Repositorios del Tercer Corte

Estructura de Datos – C++

1. ¿Qué se pide?

En el tablero se indica que para el tercer corte se deben entregar **dos repositorios**. Cada uno debe contener un **documento** (explicación teórica) y el **código en C++** correspondiente:

Repositorio	Tema	Entregables
1	Vectores y matrices en C++	Documento + código C++
2	Funciones y registros (struct) en C++	Documento + código C++

Sugerencia de estructura para cada repositorio:

- `README.md`: descripción breve y cómo compilar/ejecutar.
- `docs/`: el documento (PDF o Word) con la explicación del tema.
- `src/`: archivos `.cpp` con los ejemplos y ejercicios.

2. Vectores y matrices en C++

2.1 Vectores (arreglos unidimensionales)

Un **vector** (o arreglo) es una colección de elementos del **mismo tipo** guardados en posiciones contiguas de memoria. Se accede a cada elemento por su **índice**, que en C++ empieza en **0**.

- Se declara con tamaño fijo: `int notas[5];`
- El primer elemento es `notas[0]` y el último `notas[4]`.
- Se recorren casi siempre con un ciclo `for`.
- Acceder fuera del rango (por ejemplo `notas[5]`) es un error común y produce comportamiento indefinido.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    int notas[5];
    int suma = 0;

    for (int i = 0; i < n; i++) {
        cout << "Nota " << i + 1 << ": ";
        cin >> notas[i];
        suma += notas[i];
    }
    cout << "Promedio: " << (float)suma / n << endl;
    return 0;
}
```

Operaciones típicas con vectores: recorrer, sumar, buscar el mayor o menor, buscar un valor (búsqueda lineal) y ordenar (por ejemplo, burbuja).

2.2 Matrices (arreglos bidimensionales)

Una **matriz** es un arreglo de dos dimensiones: se organiza en **filas** y **columnas**. Se accede con dos índices: `matriz[fila][columna]`. Para recorrerla se usan **dos ciclos anidados**: el externo para las filas y el interno para las columnas.

```
#include <iostream>
using namespace std;

int main() {
    const int F = 3, C = 3;
    int m[F][C];

    // Llenar la matriz
    for (int i = 0; i < F; i++)
        for (int j = 0; j < C; j++)
            cin >> m[i][j];

    // Suma de la diagonal principal
    int diag = 0;
    for (int i = 0; i < F; i++)
        diag += m[i][i];

    // Imprimir
    for (int i = 0; i < F; i++) {
        for (int j = 0; j < C; j++)
            cout << m[i][j] << "\t";
        cout << endl;
    }
    cout << "Diagonal: " << diag << endl;
    return 0;
}
```

Operaciones típicas con matrices: suma de matrices, suma por filas o columnas, diagonal principal, transpuesta y multiplicación.

3. Funciones y registros (struct) en C++

3.1 Funciones

Una **función** es un bloque de código con nombre que realiza una tarea concreta y puede reutilizarse. Permite dividir un programa grande en partes pequeñas, más fáciles de entender, probar y mantener.

- **Tipo de retorno:** lo que devuelve (`int`, `float`, `void` si no devuelve nada).
- **Parámetros:** los datos que recibe.
- **Paso por valor:** se copia el dato; la función no modifica el original.
- **Paso por referencia** (`&`): la función trabaja con el original y puede modificarlo. Los arreglos siempre se pasan por referencia.
- **Prototipo:** declaración de la función antes del `main`.

```
#include <iostream>
using namespace std;

// Prototipos
int sumar(int a, int b);
void intercambiar(int &a, int &b);
float promedio(int v[], int n);

int main() {
    int x = 4, y = 9;
    cout << "Suma: " << sumar(x, y) << endl;

    intercambiar(x, y);              // ahora x = 9, y = 4
    cout << x << " " << y << endl;

    int datos[3] = {3, 4, 5};
    cout << "Promedio: " << promedio(datos, 3) << endl;
    return 0;
}

int sumar(int a, int b) { return a + b; }

void intercambiar(int &a, int &b) {
    int aux = a;
    a = b;
    b = aux;
}

float promedio(int v[], int n) {
    int s = 0;
    for (int i = 0; i < n; i++) s += v[i];
    return (float)s / n;
}
```

3.2 Registros (struct)

Un **registro** o `struct` agrupa varios datos, de **tipos distintos**, bajo un solo nombre. Es ideal para representar una entidad, como un estudiante, un producto o un empleado. A cada dato interno se le llama **campo** y se accede con el operador punto (`.`).

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Estudiante {
    string nombre;
    int    codigo;
    float  nota;
};

void mostrar(Estudiante e) {
    cout << e.codigo << " - " << e.nombre << ": " << e.nota << endl;
}

int main() {
    Estudiante e1 = {"Ana", 101, 4.5};
    mostrar(e1);

    // Vector de registros
    Estudiante curso[3];
    for (int i = 0; i < 3; i++) {
        cout << "Nombre: ";  cin >> curso[i].nombre;
        cout << "Codigo: ";  cin >> curso[i].codigo;
        cout << "Nota: ";    cin >> curso[i].nota;
    }
    for (int i = 0; i < 3; i++) mostrar(curso[i]);
    return 0;
}
```

Combinar todo lo visto — **vectores de registros** procesados con **funciones** — es la base de programas como un registro de estudiantes o un inventario.

4. Qué incluir en el documento de cada repositorio

- **Introducción:** qué es el tema y para qué sirve.
- **Conceptos clave:** definiciones, sintaxis y reglas importantes.
- **Ejemplos explicados:** fragmentos de código con comentarios sobre lo que hace cada parte.
- **Ejercicios resueltos:** el enunciado, el código y una captura de la ejecución.
- **Conclusiones:** lo aprendido y las dificultades encontradas.

5. Consejos finales

- Comenta el código y usa nombres de variables claros.
- Prueba cada programa con varios casos antes de subirlo.
- Haz commits frecuentes con mensajes descriptivos.
- Verifica que el repositorio compile desde cero y que el README explique cómo hacerlo.
- Confirma con tu docente el formato exacto del documento y la fecha de entrega.